

PYTHON

1. ETL Pipeline Simulation

Scenario: You're building a mini ETL pipeline for product reviews.

Given: A CSV file with columns: `review_id`, `user_id`, `rating`, `review_text`, `review_date`.

Task:

- Extract: Load the file into a DataFrame.
- Transform:
 - Remove reviews with null `rating` or `review_text`.
 - Convert `rating` to integer and filter out those < 1 or > 5 .
 - Extract sentiment using simple logic: if the text contains "bad", mark as Negative; else Positive.
- Load: Output the cleaned and enriched data to a new CSV or JSON file

2. Data Deduplication

Scenario: A CSV contains user data but has duplicates based on `email`.

Task:

- Read the file.
- Drop duplicates, keeping the most recent entry based on `last_updated` column.
- Output the cleaned file.

SQL

3. Data Joins and Filtering

Scenario: You have two tables:

- `users(user_id, name, signup_date)`
- `orders(order_id, user_id, amount, order_date)`

Task:

- Write a query to return all users who have made **more than 3 orders** in the **last 90 days**, along with their total spend.

4. Identify Data Gaps

Scenario: You're tracking hourly sensor data in:

- `sensor_logs(sensor_id, timestamp, reading)`

Task:

- Write a query to find any **missing hourly readings** in the past 24 hours for each sensor.

Scenario: Customer Purchases & Memberships

Background:

You work with a retail analytics team. They want to identify **VIP customers** based on their purchase behavior **and** membership status.

You are given two datasets:

Table 1: **purchases.csv**

transaction_id	customer_id	amount	transaction_date
T1001	C001	120.50	2024-03-15
T1002	C002	200.00	2024-03-15
T1003	C001	300.00	2024-03-16
T1004	C003	50.00	2024-03-17

Table 2: **members.csv**

customer_id	member_since	membership_level
C001	2023-05-01	Gold
C002	2023-11-15	Silver
C004	2024-01-10	Gold

Assignment Task

Objective:

Find customers who:

1. **Made total purchases over \$250** across all transactions.

2. Are Gold members.

Return:

- `customer_id`
- `total_spent`
- `membership_level`

Expected Logic:

The solution must:

- **Join** `purchases` and `members` on `customer_id`.
- **Aggregate** total purchase amount per customer.
- **Filter** only those who are **Gold** members and spent more than **\$250**.

Solve in Both:

SQL:

- Use `INNER JOIN` between the two tables.
- Use `GROUP BY`, `SUM()`, and `HAVING`.

Python (with Pandas):

- Read both CSVs.
- Merge dataframes.
- Group by `customer_id`.
- Filter based on conditions.